

Lab 7: Event-Driven Architecture (EDA) & Integration

This lab introduces the Event-Driven Architecture (EDA) pattern, focusing on asynchronous communication between microservices. We will use a Message Broker (Redis) to decouple the **Leave Service** (Event Producer) from the **Notification Service** (Event Consumer), specifically for the task of sending notifications and emails after a new Leave Request is created.

Objectives

1. Understand the concepts of Producers, Consumers, and Message Brokers in distributed systems.
2. Set up a local Message Broker (using Redis Docker).
3. Implement a simulated **Leave Service** acting as the Event Producer.
4. Implement a simulated **Notification Service** acting as the Event Consumer.
5. Demonstrate the independent, non-blocking nature of these two services.

Tool & Technology Setup

We will use Node.js and the redis library (the team's standard Backend technology) to interact with the Redis Broker.

Tool	Purpose	Installation/Setup Guide
Redis	Message Broker to store and route events (Pub/Sub).	Open Terminal and run Docker command: <code>docker run -d --name broker-redis -p 6379:6379 redis:alpine</code>
Node.js	Core programming language runtime environment.	Ensure Node.js (v16+) is installed.
redis (npm)	Node.js client library to connect to Redis.	Run: <code>npm install redis</code>
2 Terminal Windows	To run the Producer (Leave) and Consumer (Noti) concurrently.	Have 2 Terminal tabs open in your IDE.

Activity Practice 1: Setup and Broker Connection

Goal: Ensure Redis is running and set up the basic connection configuration.

Step-by-Step Instructions:

1. **Start Redis:** Ensure you have run the Docker command from the table above.
2. **Create Directory Structure:**

```
bash
```

Show full code block

```
mkdir hrm_eda_lab
```

```
cd hrm_eda_lab
```

```
npm init -y
```

```
npm install redis
```

```
touch leave_producer.js
```

```
touch noti_consumer.js
```

Activity Practice 2: Leave Service (Event Producer)

Goal: Simulate the Leave Service successfully creating a leave request and publishing a LeaveCreatedEvent to the notification channel.

Step-by-Step Instructions & Coding Guide:

1. **Implement Producer Logic:** The Producer connects to Redis and publishes a JSON string (containing event data) to a specific "channel".

File: leave_producer.js

```
javascript
```

Show full code block

```
const { createClient } = require('redis');
```

```
// Redis connection configuration
```

```
const REDIS_URL = 'redis://localhost:6379';
```

```
const CHANNEL_NAME = 'leave_events';
```

```

async function publishLeaveEvent(leaveData) {
  const client = createClient({ url: REDIS_URL });

  client.on('error', (err) => console.log('Redis Client connection error', err));

  try {
    await client.connect();

    // Convert Object to JSON string
    const message = JSON.stringify(leaveData);

    // Publish event to the channel
    await client.publish(CHANNEL_NAME, message);
    console.log(`[x] Leave Service published event for Request ID:
    ${leaveData.leave_id}`);

    await client.disconnect();
  } catch (error) {
    console.error(`[!] Error publishing event: ${error}`);
  }
}

// Simulate the user continuously sending 3 leave requests
async function simulateUserActions() {
  console.log("Leave Service is starting...");
  for (let i = 1; i <= 3; i++) {

```

```

const leaveInfo = {
  leave_id: `REQ-${i.toString().padStart(3, '0')}`,
  user_name: `Employee ${i}`,
  manager_email: `manager${i}@company.com`,
  reason: "Annual leave",
  timestamp: new Date().toISOString()
};

await publishLeaveEvent(leaveInfo);

// Simulate a 1-second delay between request submissions
await new Promise(resolve => setTimeout(resolve, 1000));
}
}

simulateUserActions();

```

Activity Practice 3: Notification Service (Event Consumer)

Goal: Implement the Notification Service. It must continuously listen for new events from the Redis channel and process them (simulating sending an Email to the Manager).

Step-by-Step Instructions & Coding Guide:

1. **Implement Consumer Logic:** The Consumer connects, "Subscribes" to the channel, and declares a callback function to run whenever a message arrives.
File: noti_consumer.js

javascript

Show full code block

```
const { createClient } = require('redis');
```

```
const REDIS_URL = 'redis://localhost:6379';
const CHANNEL_NAME = 'leave_events';

async function startNotificationService() {
  console.log("Notification Service is connecting to Redis...");

  // Create a separate client for Subscribing
  const subscriber = createClient({ url: REDIS_URL });

  subscriber.on('error', (err) => console.log('Redis Subscriber connection error', err));

  try {
    await subscriber.connect();

    console.log(`[*] Waiting for new events on channel '${CHANNEL_NAME}'. Press
CTRL+C to exit.`);

    // Listen for events
    await subscriber.subscribe(CHANNEL_NAME, async (message) => {
      try {
        const leaveData = JSON.parse(message);
        const leaveId = leaveData.leave_id;
        const email = leaveData.manager_email;

        console.log(`\n[↓] Noti Service received event: Request ${leaveId}`);
        console.log(`  Processing Email to: ${email}...`);
```

```

// SIMULATE EMAIL SENDING DELAY (Heavy task takes 3 seconds)
await new Promise(resolve => setTimeout(resolve, 3000));

// Core Business Logic
console.log(`[✓] EMAIL SENT SUCCESSFULLY: Manager needs to approve
request ${leaveId}`);

    } catch (err) {
        console.error(`[!] Error processing JSON message:`, err);
    }
});

    } catch (error) {
        console.error(`[!] Error Subscribing: ${error}`);
    }
}

startNotificationService();

```

Activity Practice 4: Testing Asynchronous Decoupling

Goal: Observe the completely independent execution process between the two services, proving the Leave Service is not slowed down by the Email sending task.

Step-by-Step Instructions:

1. Start Consumer (Notification Service):

- *Action:* Open the 1st Terminal in the hrm_eda_lab directory.
- *Command:* node noti_consumer.js
- *Observation:* The screen prints [*] Waiting for new events on channel 'leave_events'... and waits.

2. Run Producer (Leave Service):

- *Action:* Open the 2nd Terminal.
- *Command:* node leave_producer.js
- *Observation (Terminal 2 - Producer):* It will print 3 lines of [x] Leave Service published event... extremely fast (spaced by 1s based on the simulation code) and terminate. The Leave Service **does not have to wait** 3 seconds to send the email.
- *Observation (Terminal 1 - Consumer):* It will receive the event immediately, but printing [✓] EMAIL SENT SUCCESSFULLY will take 3 seconds per request.

Conclusion: This lab proves that the Leave Service has been completely decoupled from the Notification Service. If the Email sending system (NodeMailer/SMTP) experiences a network latency of 5 seconds, the employee clicking the "Submit request" button on the Website will still see a "Success" message instantly without the page hanging. This is the core power of an Event-Driven Microservices Architecture.